Authentication Feature Architecture

1. Purpose

Authentication verifies a user's identity and grants access to protected resources.

Input:

```
Email + Password
```

Output:

```
JWT Token
```

---

2. End-to-End Login Flow

```
Client
↓
LoginRequest
↓
AuthController
↓
LoginCommand
↓
MediatR
↓
LoginCommandHandler
↓
IUserRepository
↓
UserRepository
↓
AppDbContext
↓
PostgreSQL
↓
User Entity
↓
Password Verification
↓
JWT Generation
↓
LoginResponse
```

```
↓
ApiResponse
↓
Client
```

---

3. Layer Responsibilities

a. API Layer

Responsible for:

- Receiving HTTP requests
- Creating commands
- Sending commands through MediatR
- Returning responses

Main Class:

```
AuthController
```

Does NOT:

- Access database
- Verify passwords
- Generate tokens

Those responsibilities belong to Application and Infrastructure layers.

---

b. Application Layer

Responsible for:

- Business workflows
- Authentication logic
- Validation
- CQRS

Main Classes:

```
LoginCommand
LoginCommandHandler
LoginResponse
```

The Application layer decides:

```
HOW login works
```

but does not know:

```
HOW data is stored
```

---

c. Infrastructure Layer

Responsible for:

- Database access
- JWT creation
- External services

Main Classes:

```
UserRepository
JwtTokenService
AppDbContext
```

The Infrastructure layer implements technical details.

---

d. Database Layer

Responsible for:

- Storing user records
- Returning user data

Table:

```
Users
```

---

4. AuthController

Purpose

Entry point for login requests.

Receives:

```
LoginRequest
```

Creates:

```
LoginCommand
```

Sends:

```
_mediator.Send(command)
```

Returns:

```
ApiResponse<LoginResponse>
```

---

5. MediatR

Purpose

Acts as a dispatcher.

Receives:

```
LoginCommand
```

Finds:

```
LoginCommandHandler
```

Advantages:

- Loose coupling
- Cleaner controllers
- Better maintainability

---

6. LoginCommandHandler

Purpose

Contains login business logic.

Responsibilities:

a. Retrieve user

b. Verify password

c. Generate JWT token

d. Create LoginResponse

Handler Workflow

```
Receive Command
↓
Find User
↓
Verify Password
↓
Generate JWT
↓
Return Response
```

---

7. IUserRepository

Purpose

Application layer abstraction for user data access.

Application depends on:

```
IUserRepository
```

instead of

```
UserRepository
```

Benefit:

```
Loose Coupling
```

---

8. UserRepository

Purpose

Implements IUserRepository.

Acts as the bridge between:

```
Application Layer
↓
Database
```

Uses:

```
AppDbContext
```

for database operations.

---

9. AppDbContext

Purpose

Entity Framework Core gateway to the database.

Responsibilities:

a. Track entities

b. Generate SQL

c. Execute queries

d. Map database rows to C# objects

Example

```
Users Table
↓
EF Core
↓
User Entity
```

---

10. Password Verification

Purpose

Validate user credentials.

Uses:

```
BCrypt.Verify()
```

Workflow:

```
Entered Password
↓
Hash Comparison
↓
Valid / Invalid
```

Passwords are never stored in plain text.

---

11. JWT Token Generation

Purpose

Create authentication token after successful login.

Uses:

```
IJwtTokenService
```

Implementation:

```
JwtTokenService
```

Generated token contains:

a. User Id

b. Email

c. Role

d. Expiration Time

---

12. Standardized API Response

Purpose:

Return consistent responses from all endpoints.

Format:

```
{
  "success": true,
  "message": "Login successful",
  "data": {}
}
```

Benefits:

- Consistency
- Easier frontend integration
- Predictable API contracts

---

13. Dependency Flow

```
AuthController
↓
IMediator
↓
LoginCommandHandler
↓
IUserRepository
↓
UserRepository
```

```
↓
AppDbContext
↓
PostgreSQL
```

Rule:

```
Higher Layers Depend On Abstractions
Not Implementations
```

---

14. Key Concepts Demonstrated

a. Clean Architecture

b. CQRS

c. MediatR

d. Repository Pattern

e. Dependency Injection

f. JWT Authentication

g. Entity Framework Core

h. Password Hashing

i. Standardized API Responses

j. Separation of Concerns

---

15. Interview Summary

Question:

"Explain your Login Architecture."

Answer:

```
The client sends a LoginRequest to AuthController.

The controller converts it into a LoginCommand and sends it through MediatR.
```

```
MediatR routes the command to LoginCommandHandler.

The handler uses IUserRepository to retrieve the user from the database.

UserRepository uses AppDbContext and EF Core to query PostgreSQL.

The handler verifies the password using BCrypt.

If valid, JwtTokenService generates a JWT token.

The response is wrapped in ApiResponse and returned to the client.

The implementation follows Clean Architecture, CQRS, Repository Pattern,
Dependency Injection, and JWT Authentication principles.
```